🩸 BloodBridge - Optimizing Lifesaving Resources using AWS RDS & EC2

Prepared For

SmartInternz

Cloud Practitioner Guided Project

Prepared By

Siddhesh Patil

Bachelor of Engineering (Computer Science & Engineering)

Submission Date

June 2026

Abstract

BloodBridge is a cloud-based blood management system developed to streamline the complete lifecycle of blood donation, inventory management, emergency blood requests, and blood distribution. The platform connects hospitals, blood banks, and blood donors through a centralized web application deployed on Amazon Web Services (AWS).

The system addresses common challenges faced by traditional blood management processes, including delayed communication, manual inventory tracking, donor coordination issues, and emergency blood shortages. BloodBridge leverages AWS EC2 for application hosting and AWS RDS MySQL for secure database management, enabling real-time inventory monitoring, emergency request processing, donor management, and analytics-driven decision making.

By digitizing blood management operations and providing a scalable cloud-based infrastructure, BloodBridge improves operational efficiency, reduces blood wastage, and enhances emergency responsiveness. The platform demonstrates how cloud computing can be effectively utilized to solve critical healthcare resource management challenges while ensuring reliability, security, and accessibility.

Table of Contents

1. Introduction

Blood is one of the most essential healthcare resources required for surgeries, accident recovery, trauma care, cancer treatment, and emergency medical procedures. Despite technological advancements in healthcare, many hospitals and blood banks still rely on fragmented systems and manual processes to manage blood inventories and donor information.

These traditional approaches often result in delayed emergency responses, inefficient donor coordination, lack of inventory visibility, and blood wastage. In critical situations, such delays can directly impact patient survival.

BloodBridge is designed to address these challenges by providing a centralized cloud-based platform that enables seamless interaction between hospitals, blood banks, and blood donors. The platform supports real-time blood inventory tracking, emergency request management, donor registration, and data analytics.

Built using Python Flask and MySQL and deployed on AWS infrastructure, BloodBridge offers a secure, scalable, and highly available solution for modern blood resource management.

---

2. Purpose

The primary objective of BloodBridge is to create a centralized platform for managing blood resources efficiently and effectively.

The system aims to:

- Improve emergency blood availability.
- Reduce delays in blood request processing.
- Enhance communication between hospitals and blood banks.
- Improve donor engagement and participation.
- Minimize blood wastage.
- Provide secure and role-based access control.
- Deliver real-time inventory visibility and reporting.

3. Scope

The scope of BloodBridge includes:

Donor Management

- Donor registration and profile creation.
- Donation scheduling.
- Eligibility tracking.
- Donation history management.

Hospital Management

- Emergency blood request creation.
- Priority assignment for requests.
- Request tracking and monitoring.
- Patient-related request management.

Blood Bank Management

- Blood inventory updates.
- Stock monitoring.
- Shortage detection.
- Inventory reporting.

Cloud Infrastructure

- AWS EC2 application hosting.
- AWS RDS database management.
- Secure authentication and session handling.
- Centralized data storage and access.

---

4. Significance of the Project

BloodBridge provides a practical solution to healthcare organizations by improving the efficiency of blood management operations.

The project contributes through:

- Faster emergency response.
- Improved inventory utilization.
- Better donor engagement.
- Reduced blood wastage.
- Secure cloud-based operations.
- Data-driven decision making.

The platform ultimately supports the broader goal of saving lives through timely access to blood resources.

---

5. Problem Statement

Existing Challenges

Problem Statement 1

Field	Description
Customer	Hospital Administrator
Goal	Request blood during emergencies
Challenge	Difficulty identifying available blood quickly
Cause	Lack of real-time inventory visibility

Problem Statement 2

Field	Description
Customer	Blood Donor
Goal	Manage donation schedule
Challenge	Lack of donation reminders
Cause	Manual donor management
Impact	Reduced donor participation

Problem Statement 3

Field	Description
Customer	Blood Bank Manager
Goal	Manage blood inventory efficiently
Challenge	Inventory updates are not centralized
Cause	Fragmented management systems
Impact	Increased risk of shortages and wastage

6. Project Initialization and Planning Phase

Project Goals

- Develop a centralized blood management platform.
- Implement secure authentication.
- Enable real-time inventory management.
- Support emergency blood requests.
- Deploy the solution on AWS Cloud.

Development Methodology

The project followed an iterative development approach consisting of:

- Requirement Analysis
- System Design
- Database Design
- Backend Development
- Frontend Development
- AWS Deployment

---

7. Proposed Solution

BloodBridge provides a cloud-native solution that integrates hospitals, blood banks, and donors into a unified platform.

The proposed solution offers:

- Real-time blood inventory management.
- Emergency request processing.
- Donor registration and scheduling.
- Inventory analytics and reporting.
- Cloud-hosted infrastructure.

Expected Benefits

- Improved blood availability.
- Reduced response time.
- Enhanced coordination.
- Lower operational overhead.
- Better resource utilization.

---

8. System Architecture Document

Application Layer

The application layer consists of a Python Flask web application hosted on AWS EC2.

Responsibilities include:

- Authentication
- Request Processing
- Dashboard Rendering
- Data Validation

Data Layer

AWS RDS MySQL stores:

- User Accounts
- Donor Profiles
- Blood Inventory
- Emergency Requests
- Donation Records

Presentation Layer

The frontend uses: HTML, CSS5, JAVASCRIPT, BOOTSTRAP

---

9. System Architecture Diagram

Client Browser
(Desktop / Mobile)

|
▼

AWS EC2 Instance
(Ubuntu 22.04)

|
▼

Gunicorn Server

|
▼

Flask Application

|
▼

SQLAlchemy ORM

|
▼

AWS RDS MySQL
(Database Layer)

---

10. Technology Stack

Backend

- Python 3.9+
- Flask
- Flask-SQLAlchemy
- Flask-Login
- Flask-Bcrypt
- PyMySQL

Frontend

- HTML5
- CSS3
- Bootstrap 5
- JavaScript
- Chart.js
- Font Awesome

Database

- MySQL 8.0
- AWS RDS

Cloud Infrastructure

- AWS EC2
- AWS RDS
- Ubuntu 22.04
- Gunicorn

---

11. AWS Services Utilized

Amazon EC2

Used to host the Flask web application and manage application execution.

Benefits

- Scalable compute resources
- Remote deployment
- High availability
- Secure access through SSH

Amazon RDS MySQL

Used to store and manage application data.

Benefits

- Automated backups
- Managed database service
- Enhanced security
- High reliability

---

12. Database Design

Table	Purpose
Users	Authentication and authorization
Donors	Donor profile information
Inventory	Blood stock records

Emergency_Requests	Request management
Donations	Donation history
Notifications	User alerts

13. Key Features

Hospital Administrator Features

- Emergency blood requests
- Analytics dashboard
- Search and filtering

Blood Donor Features

- Registration
- Donation scheduling
- Eligibility tracking
- Donation history

Blood Bank Features

- Inventory management
- Shortage alerts
- Inventory reporting

Common Features

- Secure authentication
- Role-based access control
- Responsive design
- Dashboard analytics
- Notification system

14. Installation and Setup

(Insert your actual installation steps from README.)

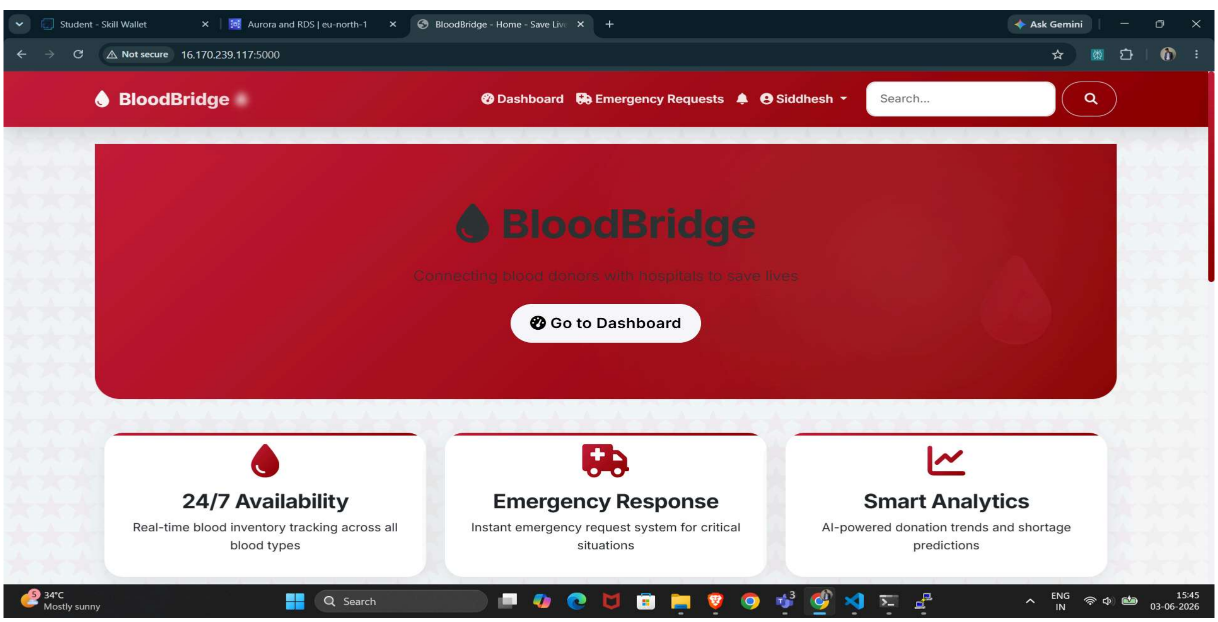
---

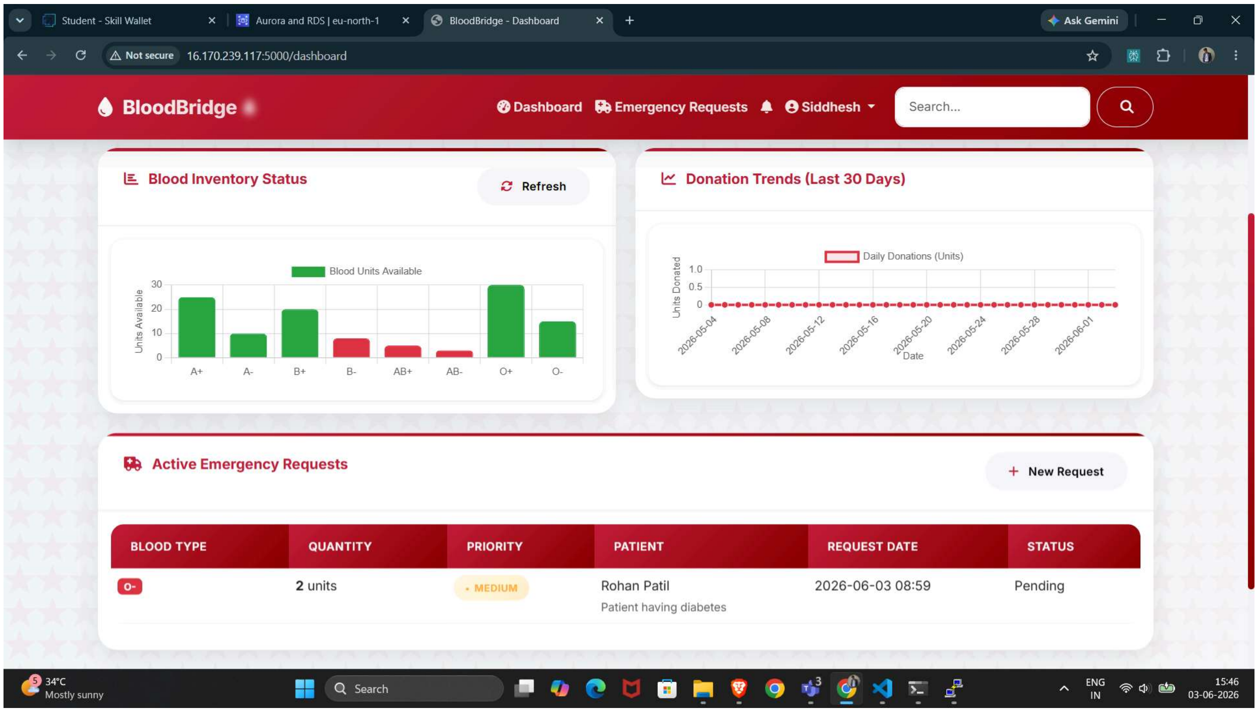
15. AWS Deployment Process

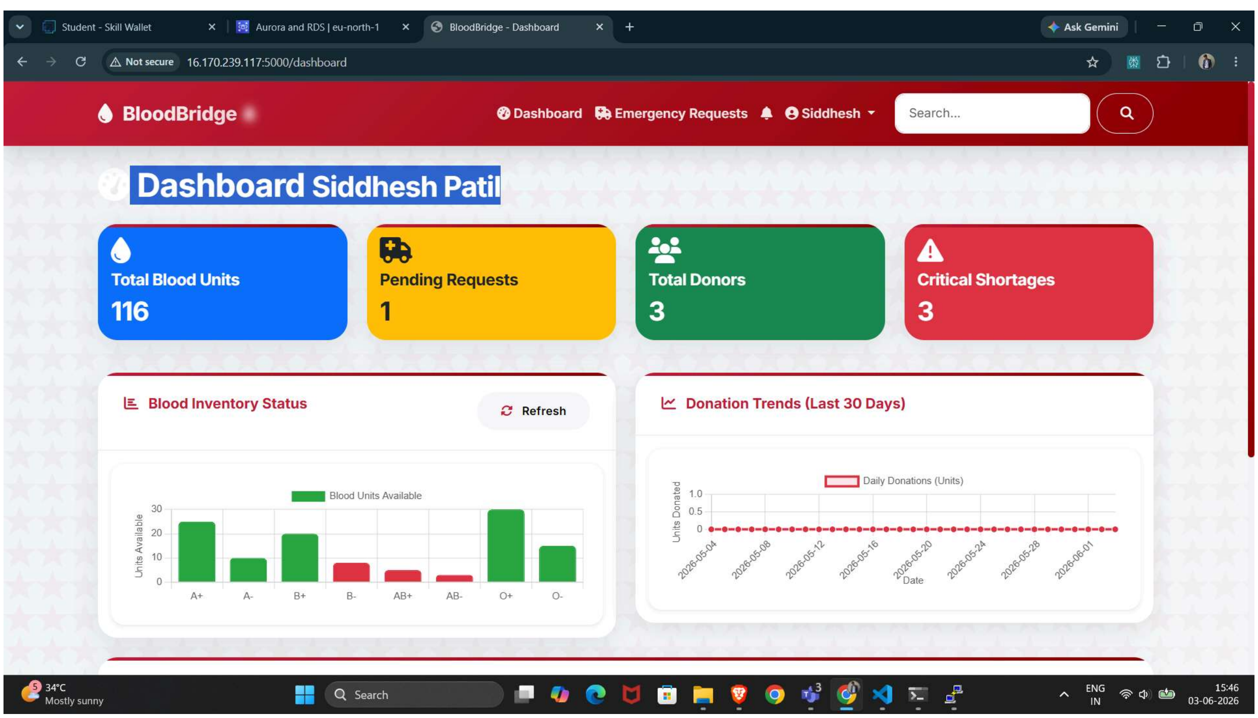
1. Create AWS EC2 instance.
2. Configure security groups.
3. Create AWS RDS MySQL database.
4. Connect EC2 to RDS.
5. Upload application files.
6. Configure environment variables.
7. Install dependencies.
8. Launch application using Gunicorn.
9. Test application connectivity.
10. Make application publicly accessible.

---

16. Results and Screenshots




Student - Skill Wallet
Aurora and RDS | eu-north-1
BloodBridge - Dashboard
Ask Gemini
Not secure
16.170.239.117:5000/dashboard
BloodBridge
Dashboard
Emergency Requests
Siddhesh
Search...
Blood Inventory Status
Refresh
Blood Units Available
Units Available
A+
A-
B+
B-
AB+
AB-
O+
O-
Donation Trends (Last 30 Days)
Daily Donations (Units)
Units Donated
Date
Active Emergency Requests
New Request
BLOOD TYPE
QUANTITY
PRIORITY
PATIENT
REQUEST DATE
STATUS
O-
2 units
MEDIUM
Rohan Patil
Patient having diabetes
2026-06-03 08:59
Pending
34°C
Mostly sunny
Search
ENG
IN
15:46
03-06-2026


Student - Skill Wallet
Aurora and RDS | eu-north-1
BloodBridge - Dashboard
Ask Gemini
Not secure
16.170.239.117:5000/dashboard
BloodBridge
Dashboard
Emergency Requests
Siddhesh
Search...
Dashboard Siddhesh Patil
Total Blood Units
116
Pending Requests
1
Total Donors
3
Critical Shortages
3
Blood Inventory Status
Refresh
Blood Units Available
Units Available
A+
A-
B+
B-
AB+
AB-
O+
O-
Donation Trends (Last 30 Days)
Daily Donations (Units)
Units Donated
Date
34°C
Mostly sunny
Search
ENG
IN
15:46
03-06-2026

17. Advantages and Disadvantages

Advantages

- Real-time inventory visibility
- Faster emergency response
- Improved donor engagement
- Secure cloud infrastructure
- Scalable architecture
- Centralized management

Disadvantages

- Internet dependency
- AWS operational costs
- Training required for new users
- Continuous maintenance needed

---

18. Conclusion

BloodBridge successfully demonstrates how cloud computing and modern web technologies can improve healthcare resource management. The platform provides a centralized ecosystem for hospitals, blood banks, and donors, enabling efficient blood inventory management and emergency response.

Through the use of AWS EC2 and AWS RDS, the application achieves scalability, security, and reliability while reducing manual effort and operational inefficiencies. BloodBridge serves as a practical example of how cloud-based solutions can positively impact healthcare operations and contribute to saving lives.

---

19. Future Scope

- Mobile Application Development
- AI-Based Blood Demand Prediction
- SMS and Email Notifications
- Multi-Hospital Integration
- Advanced Analytics Dashboard
- Blood Group Matching Algorithms
- National Blood Network Expansion

---

20. References

- AWS Documentation
- Flask Documentation
- MySQL Documentation

---

21. Appendix

Live Application

http://16.170.239.117:5000

Project Demonstration Video

https://drive.google.com/file/d/19NSMNa3Mw6lBeyeMDO_5RPmg3PQOLdP8/view?usp=drive_link

Developer

Siddhesh Patil

Full Stack Developer | AWS Cloud Practitioner | Python Flask Developer